

# Lezione 1

## Introduzione a R

Paolo Bosetti

### Indice

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>1 Documenti Quarto (estensione .qmd)</b>       | <b>2</b>  |
| 1.1 Intestazione in YAML . . . . .                | 2         |
| 1.2 Rendering: requisiti . . . . .                | 2         |
| 1.3 Formattazione: Markdown . . . . .             | 3         |
| 1.3.1 Questo è un titolo di livello 3 . . . . .   | 3         |
| 1.4 Elenchi puntati e numerati . . . . .          | 3         |
| 1.5 Notebook: unire testo e codice . . . . .      | 4         |
| <b>2 Richiamo basi di R</b>                       | <b>4</b>  |
| 2.1 Vettori . . . . .                             | 4         |
| 2.2 Matrici . . . . .                             | 7         |
| 2.3 Tabelle ( <code>data.frame</code> ) . . . . . | 9         |
| <b>3 Funzioni di statistica descrittiva</b>       | <b>11</b> |
| <b>4 Intermezzo: programmazione R</b>             | <b>12</b> |
| 4.1 Funzioni pesonali . . . . .                   | 12        |
| 4.2 Tidyverse . . . . .                           | 14        |
| 4.2.1 Pipe . . . . .                              | 15        |
| 4.2.2 Tibble . . . . .                            | 16        |
| <b>5 Statistica inferenziale</b>                  | <b>17</b> |
| 5.1 Funzioni di distribuzione . . . . .           | 17        |
| 5.2 Test di Student . . . . .                     | 18        |

# 1 Documenti Quarto (estensione .qmd)

Un documento Quarto è un file di testo che combina **testo formattato** (in Markdown) e **codice eseguibile** (in questo corso, codice R). Quando il documento viene *renderizzato*, il codice viene eseguito e i risultati (numeri, tabelle, grafici) vengono inseriti automaticamente nel documento finale (HTML, PDF, Word, ecc.).

## 1.1 Intestazione in YAML

Ogni documento inizia con un'intestazione delimitata da due linee ---, scritta in linguaggio YAML (un formato di testo del tipo **chiave: valore**). L'intestazione contiene i **metadati** del documento e le opzioni di rendering:

```
---
title: "Lezione 1"
subtitle: "Introduzione a R"
author: "Paolo Bosetti"
format: html
---
```

- `title`, `subtitle` e `author` definiscono titolo, sottotitolo e autore;
- `format` stabilisce il formato di output (`html`, `pdf`, `docx`, ...);
- altre opzioni utili sono ad esempio `toc: true` (indice dei contenuti), `date: today` (data di generazione) ed `embed-resources: true` (produce un unico file HTML autonomo).

Attenzione all'**indentazione**: in YAML è significativa, e le opzioni annidate vanno indentate con due spazi, ad esempio:

```
format:
  html:
    toc: true
```

Dopo i due punti lo spazio è **obbligatorio**!

## 1.2 Rendering: requisiti

Prima di renderizzare un documento Quarto, è necessario installare in R il *package* `rmarkdown`: dalla console di R eseguire il comando: `install.packages("rmarkdown")`.

Per creare documenti in HTML (più rapido, adatto ad una preview del contenuto), è poi sufficiente cliccare il pulsante “Preview” sulla toolbar del documento.

Per creare documenti in PDF (maggiore qualità), è necessario installare in R il package `tinytex`: dalla console di R eseguire il comando: `install.packages("tinytex")`, e successivamente eseguire `tinytex::install_tinytex()` per installare la distribuzione LaTeX completa. Questa operazione di setup va eseguita **una volta sola**. Dopodiché cliccare sul pulsante “Preview format...” e scegliere il formato “PDF (LaTeX)”

## 1.3 Formattazione: Markdown

Tipo di carattere: *corsivo*, **grassetto**, monospaziato (quest’ultimo per il codice).

### 1.3.1 Questo è un titolo di livello 3

I paragrafi vanno separati da una o più **linee vuote**.

Questo è il secondo *paragrafo*.

I commenti non compaiono nel documento finale. Per i titoli si usano uno o più # all’inizio della linea (uno per ogni livello).

## 1.4 Elenchi puntati e numerati

- Questo
- è un elenco
- puntato
  - con un
  - sottoelenco

Gli elenchi numerati si fanno con i numeri o le lettere:

1. Primo elemento
2. Secondo elemento: questo  
è una ulteriore linea (a capo grazie al carattere \)
3. Terzo elemento
  - a. Sottoelemento
  - b. Altro sottoelemento
4. Quarto elemento

## 1.5 Notebook: unire testo e codice

Si può inserire un blocco di codice (*chunk*) così:

```
40 + 2
```

```
[1] 42
```

Il risultato dell'esecuzione compare subito sotto il blocco. Le scorciatoie di tastiera utili (in RStudio e Positron) sono:

- per inserire un blocco: Windows **Ctrl+Alt+I**, Mac **Cmd+Option+I**
- per eseguire la linea attiva (o la selezione): Windows **Ctrl+Invio**, Mac **Cmd+Invio**
- per eseguire l'intero blocco: Windows **Ctrl+Maiusc+Invio**, Mac **Cmd+Maiusc+Invio**

## 2 Richiamo basi di R

### 2.1 Vettori

Assegnazioni e vettori:

```
# assegnazione (si preferisce <- a =)
x <- 5
y = 3 # equivalente, ma meno usato
# vettori: la funzione c() concatena i valori
v <- c(10, 20, 30, 40, 50)
v2 <- 1:5 # sequenza di interi
v3 <- seq(0, 1, by = 0.25)
v
```

```
[1] 10 20 30 40 50
```

```
v2
```

```
[1] 1 2 3 4 5
```

```
v3
```

```
[1] 0.00 0.25 0.50 0.75 1.00
```

```
# indicizzazione di un vettore (in R gli indici partono da 1!)  
v[2]      # secondo elemento
```

```
[1] 20
```

```
v[c(1, 3)] # primo e terzo elemento
```

```
[1] 10 30
```

```
v[-1]      # tutti tranne il primo
```

```
[1] 20 30 40 50
```

```
v[2:4]     # dal secondo al quarto
```

```
[1] 20 30 40
```

```
# indicizzazione logica e filtraggio  
v > 25      # vettore di valori logici
```

```
[1] FALSE FALSE  TRUE  TRUE  TRUE
```

```
v[v > 25]   # solo gli elementi maggiori di 25
```

```
[1] 30 40 50
```

```
v[v > 15 & v < 45] # combinazione di condizioni con & (AND) e | (OR)
```

```
[1] 20 30 40
```

Operazioni vettorializzate:

```
# operazioni vettore-scalare: lo scalare è applicato a ogni elemento  
v * 2
```

```
[1] 20 40 60 80 100
```

```
v + 1
```

```
[1] 11 21 31 41 51
```

```
v^2
```

```
[1] 100 400 900 1600 2500
```

```
# operazioni vettore-vettore e riciclo degli elementi del vettore più corto  
c(1, 2, 3, 4) + c(10, 20, 30, 40) # elemento per elemento
```

```
[1] 11 22 33 44
```

```
c(1, 2, 3, 4) + c(10, 20) # il secondo vettore viene riciclato: 10, 20, 10, 20
```

```
[1] 11 22 13 24
```

Introduciamo una funzione utile nella forma più semplice (ne vedremo il dettaglio più avanti): `sample(n)` ritorna i numeri da 1 a `n` in ordine casuale:

```
v1 <- sample(10)  
v1
```

```
[1] 1 6 9 8 2 5 4 10 7 3
```

La lunghezza di un vettore si ottiene con la funzione `length()`. Per ottenere gli ultimi tre elementi di un vettore, ad esempio:

```
v <- sample(10)  
v[length(v)-2:0]
```

```
[1] 5 10 9
```

Qui `2:0` è la sequenza 2, 1, 0, quindi gli indici sono `length(v)-2`, `length(v)-1` e `length(v)`. In alternativa, si può usare la funzione `tail(v, 3)`.

È anche possibile usare `length()` per **cambiare** la lunghezza di un vettore:

```
# allungare un vettore: i nuovi elementi sono NA (dato mancante)
length(v) <- 12
v
```

```
[1] 2 4 3 7 1 6 8 5 10 9 NA NA
```

```
# accorciare un vettore: gli elementi in eccesso vengono persi
length(v) <- 5
v
```

```
[1] 2 4 3 7 1
```

### **i** Attenzione

Le *funzioni freccia* come `length()` <- sono una particolarità di R. Non tutte le funzioni hanno l'equivalente freccia: consultare l'help in linea di una funzione per scoprire se la versione freccia (cioè con assegnazione) esiste (es: `?length`).

## 2.2 Matrici

Le matrici si creano con la funzione `matrix()`:

```
# per righe:
m1 <- matrix(1:6, nrow = 2, byrow = TRUE)
m1
```

```
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

```
# per colonne (comportamento predefinito)
m2 <- matrix(1:6, nrow = 2)
m2
```

```
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

L'indicizzazione avviene come per i vettori, ma passando a [] due indici (riga e colonna). Un indice mancante significa “intera riga/colonna”:

```
m1[2, 3]    # elemento in riga 2, colonna 3
```

```
[1] 6
```

```
m1[1, ]    # intera prima riga
```

```
[1] 1 2 3
```

```
m1[, 2]    # intera seconda colonna
```

```
[1] 2 5
```

```
m1[, c(1, 3)] # prima e terza colonna
```

```
      [,1] [,2]  
[1,]     1     3  
[2,]     4     6
```

Le operazioni aritmetiche tra matrici (+, -, \*, /) avvengono elemento per elemento:

```
m1 * m2
```

```
      [,1] [,2] [,3]  
[1,]     1     6    15  
[2,]     8    20    36
```

```
m1 + 10
```

```
      [,1] [,2] [,3]  
[1,]    11    12    13  
[2,]    14    15    16
```

Per il prodotto matriciale si usa invece %\*% (le dimensioni devono essere compatibili: colonne della prima = righe della seconda!). Per trasporre una matrice si usa t():

```
m1 %*% t(m2) # (2x3) %*% (3x2) = matrice 2x2
```

```
      [,1] [,2]
[1,]    22    28
[2,]    49    64
```

## 2.3 Tabelle (data.frame)

Più utili delle matrici sono le **tabelle**, o **data.frame**: rappresentano matrici le cui colonne possono essere eterogenee (numeri, testo, valori logici, ...) e sono indicate da un **nome**.

```
# creazione di una tabella
df <- data.frame(
  A = 1:10,
  B = sample(10),
  Lettere = LETTERS[1:10]
)
df
```

```
      A  B Lettere
1     1  5      A
2     2  6      B
3     3 10      C
4     4  1      D
5     5  2      E
6     6  4      F
7     7  3      G
8     8  8      H
9     9  9      I
10    10  7      J
```

L'indicizzazione avviene esattamente come per le matrici con `[]`, oppure estraendo un'intera colonna **per nome** come vettore:

```
df[3, ] # terza riga
```

```
      A  B Lettere
3     3 10      C
```

```
df[, 2] # seconda colonna
```

```
[1] 5 6 10 1 2 4 3 8 9 7
```

```
df[3, "B"] # elemento singolo, con la colonna indicata per nome
```

```
[1] 10
```

```
df$B # intera colonna B, come vettore
```

```
[1] 5 6 10 1 2 4 3 8 9 7
```

```
df[["B"]] # equivalente a df$B
```

```
[1] 5 6 10 1 2 4 3 8 9 7
```

Nuove colonne possono essere create assegnandole per nome:

```
df$NuovaColonna <- df$A*2  
head(df) # mostra solo le prime righe
```

|   | A | B  | Lettere | NuovaColonna |
|---|---|----|---------|--------------|
| 1 | 1 | 5  | A       | 2            |
| 2 | 2 | 6  | B       | 4            |
| 3 | 3 | 10 | C       | 6            |
| 4 | 4 | 1  | D       | 8            |
| 5 | 5 | 2  | E       | 10           |
| 6 | 6 | 4  | F       | 12           |

La colonna può essere a sua volta indicizzata, e può essere usata per filtrare le righe della tabella (indicizzazione logica):

```
df[df$B>5,]
```

|    | A  | B  | Lettere | NuovaColonna |
|----|----|----|---------|--------------|
| 2  | 2  | 6  | B       | 4            |
| 3  | 3  | 10 | C       | 6            |
| 8  | 8  | 8  | H       | 16           |
| 9  | 9  | 9  | I       | 18           |
| 10 | 10 | 7  | J       | 20           |

### 3 Funzioni di statistica descrittiva

Funzioni campionarie:

```
sum(df$B^2) # somma dei quadrati
```

```
[1] 385
```

```
mean(df$B) # media campionaria
```

```
[1] 5.5
```

```
sd(df$B) # deviazione standard campionaria
```

```
[1] 3.02765
```

```
var(df$B) # varianza campionaria
```

```
[1] 9.166667
```

La somma dei quadrati degli scarti dalla media si calcola ad esempio con  $\sum_{i=1}^N (y_i - \bar{y})^2$

```
ss <- sum((df$B - mean(df$B))^2)
ss
```

```
[1] 82.5
```

Si noti che la varianza campionaria è proprio  $ss/(N-1)$ :

```
ss / (length(df$B) - 1)
```

```
[1] 9.166667
```

```
var(df$B)
```

```
[1] 9.166667
```

Attenzione ai valori NA (dati mancanti): quasi tutte le funzioni statistiche restituiscono NA se anche un solo elemento è NA, a meno di indicare esplicitamente di ignorarli con `na.rm=TRUE`:

```
v5 <- c(3, 7, 4, NA, 9)
mean(v5) # NA
```

```
[1] NA
```

```
mean(v5, na.rm=TRUE) # ignora gli NA
```

```
[1] 5.75
```

```
mean(v5[!is.na(v5)]) # equivalente: filtra gli NA a mano
```

```
[1] 5.75
```

```
length(v5[!is.na(v5)]) # numero di dati validi
```

```
[1] 4
```

## 4 Intermezzo: programmazione R

### 4.1 Funzioni pesonali

Come si crea una funzione? le funzioni di R si creano con la parola chiave `function()`. Ad esempio, una funzione che moltiplica due numeri:

```
my_f <- function(a, b = 2) {
  return(a * b)
}
```

Se il blocco di una funzione ha una sola istruzione, si possono omettere le parentesi graffe. Inoltre, ogni funzione restituisce automaticamente l'ultimo valore calcolato, quindi si può omettere anche `return()`. Quindi le due definizioni seguenti sono equivalenti a quella sopra:

```
my_f <- function(a, b = 2) return(a * b)
my_f <- function(a, b = 2) a * b
```

### Nota

Come si vede, per essere usata una funzione deve essere **assegnata a una variabile** (in questo caso `my_f`). In R le funzioni sono **oggetti di prima classe**, quindi possono essere passate come argomenti ad altre funzioni, restituite come valori da altre funzioni, assegnate a variabili, ecc.

È ovviamente anche possibile ri-assegnare una funzione, come fatto nell'ultimo blocco.

Vediamo un esempio più complesso: una funzione che calcola la somma di un vettore. La funzione `length()` restituisce la lunghezza di un vettore, e il ciclo `for` permette di iterare su tutti gli elementi del vettore.

```
my_sum <- function(v) {  
  s <- 0  
  for (i in 1:length(v)) {  
    s <- s + v[i]  
  }  
  return(s)  
}  
  
my_sum(1:3)
```

```
[1] 6
```

Il loop `for` può anche iterare direttamente sugli elementi del vettore, senza usare gli indici: vediamo come fare, aggiungendo anche il controllo sui valori NA (dati mancanti):

```
my_sum <- function(v, na.rm = FALSE) {  
  s <- 0  
  for (e in v) {  
    if (!is.na(e)) s <- s + e  
  }  
  return(s)  
}  
  
my_sum(c(1:3, NA))
```

```
[1] 6
```

Oltre alla struttura condizionale `if|else`, è disponibile anche la funzione `ifelse()`

```
my_sum <- function(v, na.rm = FALSE) {  
  s <- 0  
  for (e in v) s <- s + ifelse(is.na(e), 0, e)  
  return(s)  
}  
  
my_sum(c(1:3, NA))
```

```
[1] 6
```

Spesso succede di creare funzioni che possono essere ri-utilizzate in differenti documenti. In questo caso è utile salvare le funzioni in un file separato, detto *script* o *libreria*, e richiamarlo con la funzione `source()`. In questo modo le funzioni definite nel file esterno diventano disponibili nel documento corrente.

#### ! Importante

L'estensione dei file script di R è `.R`, mentre l'estensione dei documenti Quarto è `.qmd`. In questo corso, il file `my_functions.R` contiene le funzioni `my_f()` e `my_sum()`, che sono richiamate nel documento `lezione1.qmd`.

Per caricare il nostro file “libreria”:

```
source("my_functions.R")
```

#### 💡 Consiglio

Di solito è opportuno caricare librerie e pacchetti all'inizio del documento, in un blocco di setup, come fatto all'inizio di questo documento. In questo modo le funzioni definite nel file esterno diventano disponibili in tutti i blocchi di codice del documento.

## 4.2 Tidyverse

Introduciamo l'uso di una libreria (*package*) molto utile e che sarà utilizzata estensivamente durante il corso: **Tidyverse**. Tidyverse è un insieme di pacchetti R che condividono una filosofia comune e sono progettati per lavorare bene insieme. La introdurremo gradualmente, ma per ora ci limitiamo a installarla e caricarla all'inizio del documento con `library(tidyverse)`. In questo modo tutte le funzioni dei pacchetti Tidyverse diventano disponibili nel documento.

### ! Importante

Per installare Tidyverse, nella console di R eseguire `install.packages("tidyverse")` (è necessario essere connessi a Internet). Una volta installata, la libreria va caricata all'inizio di ogni documento con `library(tidyverse)`.

## 4.2.1 Pipe

La prima funzionalità di Tidyverse che incontriamo è la *pipe* `%>`, che permette di concatenare più funzioni in sequenza, rendendo più chiara l'esecuzione di più funzioni in sequenza.

Infatti, spesso succede di dover annidare funzioni, cioè usare il risultato di una funzione come argomento di un'altra funzione. Ad esempio potrei scrivere:

```
my_f(my_sum(c(1,3,2)), b=3)
```

```
[1] 18
```

L'espressione è corretta, ma non è molto chiara né leggibile: la prima funzione ad essere chiamata è infatti la più interna, e è necessario contare bene le parentesi per essere sicuri che i vari parametri siano passati alle funzioni giuste.

Per rendere più chiara l'espressione potrei usare variabili intermedie:

```
v <- c(1,3,2)
v_s <- my_sum(v)
v_r <- my_f(v_s, b=3)
```

Tuttavia, ciò “sporca” l'ambiente definendo molte variabili temporanee che non servono più dopo l'esecuzione e che pure occupano memoria (inutilmente!).

La *pipe* rende più chiara l'esecuzione di più funzioni in sequenza. Essa passa l'output della funzione a sinistra come primo argomento della funzione a destra. Quindi l'esempio precedente può essere scritto così:

```
c(1,3,2) %>%
  my_sum() %>%
  my_f(b=3)
```

```
[1] 18
```

Se voglio salvare il risultato, ovviamente:

```
res <- c(1,3,2) %>%  
  my_sum() %>%  
  my_f(b=3)
```

### ! Scorciatoia

Per inserire la pipe in RStudio, si può usare la scorciatoia **Ctrl + Shift + M** (o **Cmd + Shift + M** su macOS). Tuttavia attenzione: in R ci sono **due pipe**: la **pipe di Tidyverse %>%** e la **pipe base |>**. La prima è più potente e flessibile, ma non è nativa di R (è definita nel pacchetto **magrittr**, che fa parte di Tidyverse). La seconda è nativa di R, ma meno potente. In questo corso useremo la pipe di Tidyverse.

Per fare in modo che la scorciatoia da tastiera inserisca la pipe di Tidyverse, in RStudio aprire i setting di Positron con **Ctrl + ,** (o **Cmd + ,** su macOS), cercare “pipe” e selezionare **%>%** come scorciatoia.

## 4.2.2 Tibble

Un'altra struttura dati importante in Tidyverse è la **tibble**. Una **tibble** è una variante evoluta di un **data.frame**. Si crea una tibble con la funzione **tibble()**, che accetta come argomenti i nomi delle colonne e i valori corrispondenti, esattamente come **data.frame**.

In più, però, consente di definire una colonna come funzione di altre colonne, definite prima. Ad esempio:

```
tibble(  
  A = 1:10,  
  B = A * 2 # Questo NON FUNZIONEREBBE con data.frame()!  
)
```

```
# A tibble: 10 x 2
```

|   | A     | B     |
|---|-------|-------|
|   | <int> | <dbl> |
| 1 | 1     | 2     |
| 2 | 2     | 4     |
| 3 | 3     | 6     |
| 4 | 4     | 8     |
| 5 | 5     | 10    |
| 6 | 6     | 12    |
| 7 | 7     | 14    |

|    |    |    |
|----|----|----|
| 8  | 8  | 16 |
| 9  | 9  | 18 |
| 10 | 10 | 20 |

## 5 Statistica inferenziale

### 5.1 Funzioni di distribuzione

In generale R dispone di numerose funzioni di distribuzione, i cui nomi seguono questo pattern:  
`[r|p|q|d] [nome] ()`

Ad esempio, per la distribuzione normale, le funzioni sono

- `rnorm()`: genera numeri casuali normali
- `pnorm()`: probabilità di un dato valore (CDF)
- `qnorm()`: quantile di una data probabilità
- `dnorm()`: PDF di un dato valore

Le distribuzioni più comuni sono:

- `norm`: normale
- `unif`: uniforme
- `t`: T di Student
- `chisq`: Chi quadro
- `f`: F di Snedecor
- `binom`: binomiale (discreta)
- `pois`: Poisson (discreta)
- `geom`: geometrica (discreta)

Ad esempio, per generare 10 valori casuali da una distribuzione normale con media 3 e deviazione standard 5:

```
set.seed(1)
rnorm(10, mean=3, sd=5)
```

```
[1] -0.1322691  3.9182166 -1.1781431 10.9764040  4.6475389 -1.1023419
[7]  5.4371453  6.6916235  5.8789068  1.4730581
```

Si noti che `set.seed()` serve a fissare il seme del generatore di numeri casuali, in modo che i risultati siano riproducibili. Se si esegue più volte lo stesso codice senza fissare il seme, i valori generati saranno diversi ogni volta:

```
rnorm(10, mean=3, sd=5)
```

```
[1] 10.5589058  4.9492162 -0.1062029 -8.0734994  8.6246546  2.7753320  
[7]  2.9190487  7.7191811  7.1061060  5.9695066
```

Per le funzioni PDF, CDF e quantile, ritardiamo l'illustrazione a quando avremo imparato a realizzare grafici in R, in modo da poter visualizzare le distribuzioni.

## 5.2 Test di Student

Vediamo come realizzare un test di Student per confrontare due campioni. Creiamo due campioni casuali e applichiamo la funzione `t.test()`:

```
set.seed(1)  
c1 <- rnorm(12, 10, 1.1)  
c2 <- rnorm(10, 15.2, 1.1)  
  
t.test(c1, c2)
```

Welch Two Sample t-test

```
data:  c1 and c2  
t = -11.744, df = 17.068, p-value = 1.336e-09  
alternative hypothesis: true difference in means is not equal to 0  
95 percent confidence interval:  
 -6.082240 -4.230104  
sample estimates:  
mean of x mean of y  
 10.29550  15.45167
```

### ! Attenzione!

Di default, `t.test()` assume che i due campioni siano eteroschedastici, e effettua quindi un test di Welch! consultare l'help in linea di `t.test()` per maggiori dettagli. Per forzare l'ipotesi di varianze uguali, usare l'opzione `var.equal=TRUE`.

Per un test a un campione è opportuno specificare il valore di  $\mu_0$  nell'ipotesi nulla, con l'opzione `mu`:

```
t.test(c1, mu=9)
```

#### One Sample t-test

```
data: c1
t = 5.0322, df = 11, p-value = 0.0003826
alternative hypothesis: true mean is not equal to 9
95 percent confidence interval:
 9.728879 10.862124
sample estimates:
mean of x
 10.2955
```

Per verificare l'omoschedasticità dei due campioni, si può utilizzare il test della varianza:

```
var.test(c1,c2)
```

#### F test to compare two variances

```
data: c1 and c2
F = 0.62869, num df = 11, denom df = 9, p-value = 0.4618
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.160704 2.255662
sample estimates:
ratio of variances
 0.6286861
```

Il  $p$ -value grande consiglia di accettare l'ipotesi nulla di varianze uguali, quindi possiamo forzare l'ipotesi di varianze uguali nel test di Student:

```
t.test(c1, c2, var.equal=T)
```

#### Two Sample t-test

```
data: c1 and c2
t = -12.002, df = 20, p-value = 1.355e-10
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -6.052308 -4.260036
```

```
sample estimates:
mean of x mean of y
10.29550  15.45167
```

### Esercizio

Provare a cambiare gli argomenti `var.equal`, `mu`, e `conf.level` di `t.test()` osservando come cambia l'output e cercando di commentare le differenze.

Le funzioni di test oltre che stampare il report ritornano anche un oggetto risultato che può essere salvato per accedere programmaticamente ai valori di interesse (statistica del test, p-value, intervallo di confidenza, ecc.). Ad esempio:

```
vt <- var.test(c1,c2)
attributes(vt) # mostra il contenuto di vt
```

```
$names
[1] "statistic"  "parameter"  "p.value"    "conf.int"   "estimate"
[6] "null.value" "alternative" "method"     "data.name"

$class
[1] "htest"
```

In questo modo possiamo accedere al p-value del test della varianza con `vt$p.value`. Possiamo quindi usare questo valore per decidere se forzare o meno l'ipotesi di varianze uguali nel test di Student:

```
t.test(c1, c2, var.equal=(vt$p.value > 0.05))
```

### Two Sample t-test

```
data:  c1 and c2
t = -12.002, df = 20, p-value = 1.355e-10
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -6.052308 -4.260036
sample estimates:
mean of x mean of y
10.29550  15.45167
```

### Oggetti composti e liste

Le funzioni di test ritornano un oggetto che è in realtà una **lista**. Una lista è una collezione eterogenea di valori e oggetti identificati per nome.

Una lista può essere creata con la funzione `list()`:

```
l <- list(n1=10, n2=1:4, n3="test", n4=vt)
```

Per accedere ai contenuti della lista si usa lo stesso operatore `$` che abbiamo visto per le tabelle: `l$n1` restituisce il numero 10, `l$n2` restituisce il vettore `1:4`, `l$n3` restituisce la stringa `"test"`, e `l$n4` restituisce l'oggetto risultato del test della varianza. Per accedere al p-value del test della varianza, quindi, possiamo scrivere:

```
l$n4$p.value
```

```
[1] 0.4617866
```